Nama : Hasan Zubeir Pohan
NIM : 714240045
Kelas : 3C
Matkul : Pemrograman 4
Tanggal Praktikum : 21 Oktober 2026

JURNAL PRAKTIKUM PEMROGRAMAN IV

Modul 05 – Pemrograman Berorientasi Objek Lanjut (Pewarisan, Polimorfisme, Kelas Abstrak, dan Mixin)

1. Tujuan Praktikum

Mahasiswa mampu menerapkan pewarisan, polimorfisme, dan abstraksi pada program aplikasi logistik SiLog (CPMK-02). Secara khusus, praktikum ini bertujuan untuk:

- Menyusun hierarki kelas layanan pengiriman dengan kelas abstrak `LayananPengiriman` sebagai kerangka dan kelas turunan untuk tiap moda transportasi.
- Menerapkan penggantian metode (`@override`) dan keyword `super` pada perhitungan tarif dan estimasi waktu tiap moda.
- Menggunakan polimorfisme, yaitu memanggil metode melalui tipe induk `List<LayananPengiriman>` sehingga perilaku kelas turunan yang dijalankan.
- Menggunakan mixin `Terlacak` dan `Terasuransi` untuk berbagi perilaku antar kelas yang tidak berada pada satu jalur pewarisan.
- Menambahkan moda baru `PengirimanUdara` tanpa mengubah kode pemanggil, serta memasang mixin `Terasuransi` hanya pada moda kapal dan udara.

2. Alat dan Bahan

No	Alat / Bahan	Keterangan
1	Flutter SDK & Dart SDK	Flutter 3.47.4 (channel stable); Dart SDK bawaan Flutter (terpasang pada Modul 1)
2	Visual Studio Code	Dengan ekstensi Flutter dan Dart
3	Terminal PowerShell	Menjalankan `dart run` dan perintah Git
4	draw.io (diagrams.net)	Menggambar diagram hierarki kelas sebelum menulis kode
5	Git & GitHub	https://github.com/Nogi3201/silog-714240045/tree/main/Pertemuan%205
6	Proyek silog_app	Proyek yang sama dari modul sebelumnya

3. Langkah Kerja

A. Penyiapan

Proyek `silog_app` dibuka melalui File > Open Folder pada Visual Studio Code, kemudian dibuka terminal terintegrasi. Cabang kerja dibuat dengan `git checkout -b praktikum/modul-05` dan cabang aktif dipastikan melalui `git branch`. Direktori `lib/layanan` dibuat pada panel Explorer. Sebelum menulis kode, hierarki kelas digambar terlebih dahulu (Gambar 1).

B. Kelas Abstrak dan Mixin

Berkas `lib/layanan/layanan_pengiriman.dart` dibuat dan diisi dengan mixin `Terlacak` serta kelas abstrak `LayananPengiriman` (Kode Program 5.1).

C. Kelas Turunan Moda Transportasi

Berkas `lib/layanan/moda.dart` dibuat dan diisi dengan tiga kelas turunan, yaitu `PengirimanTruk`, `PengirimanKeretaApi`, dan `PengirimanKapal`, masing-masing memakai mixin `Terlacak` (Kode Program 5.2).

D. Pengujian Polimorfisme

Berkas `latihan/modul05_uji.dart` dibuat dan dijalankan dengan `dart run latihan/modul05_uji.dart`. Tarif keempat layanan dibandingkan dan penyebab perbedaannya dianalisis berdasarkan rumus tiap kelas. Sebagai eksperimen, objek `LayananPengiriman(...)` dibentuk langsung untuk mengamati pesan error compiler, kemudian baris tersebut dihapus kembali.

E. Latihan Penambahan Moda Baru

Kelas `PengirimanUdara` ditambahkan pada `moda.dart` dengan mewarisi `LayananPengiriman` dan memakai mixin `Terlacak`. Tarifnya memperhitungkan berat volumetrik dan biaya bandara tetap. Satu objek `UDR-CARGO` ditambahkan ke `daftarLayanan`, program dijalankan ulang, dan dipastikan tidak ada baris kode lain yang diubah.

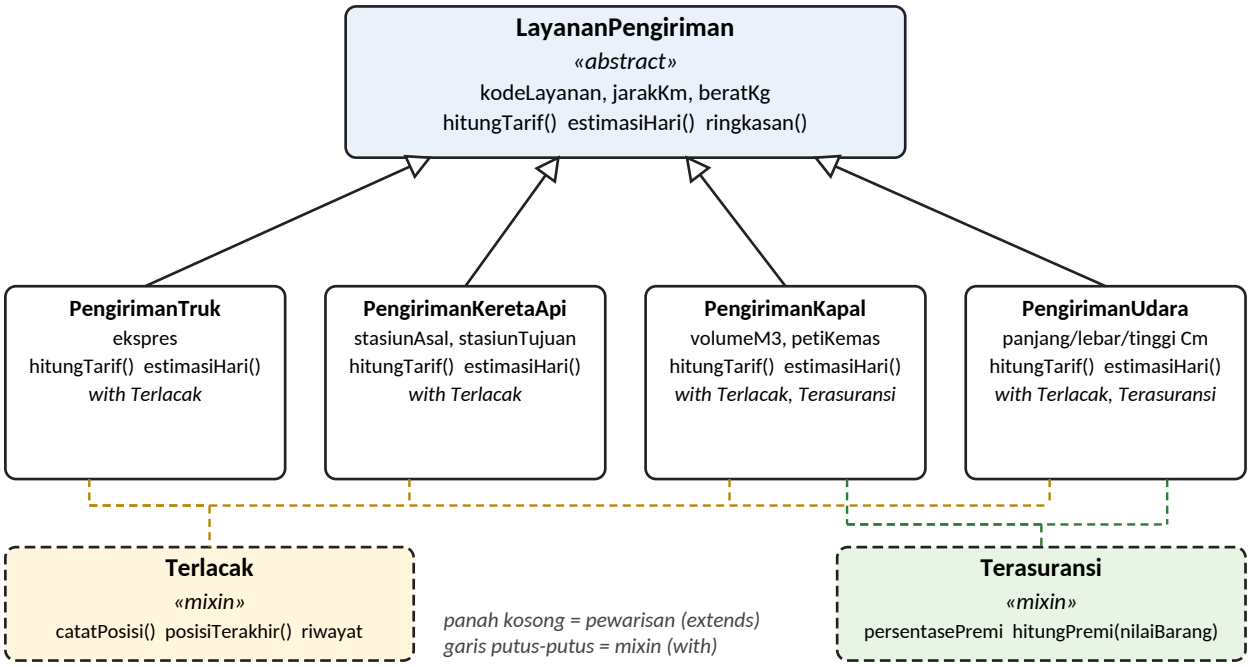
F. Tugas Praktikum

Mixin `Terasuransi` dibuat dan dipasang hanya pada `PengirimanKapal` dan `PengirimanUdara`. Program uji ditambah bagian pengujian mixin untuk menunjukkan bahwa `PengirimanTruk` tidak memiliki metode `hitungPremi()`.

4. Kode Program dan Penjelasan

4.1 Diagram hierarki kelas

Gambar 1 menunjukkan hierarki akhir. `LayananPengiriman` adalah kelas abstrak yang menyimpan atribut bersama (`kodeLayanan`, `jarakKm`, `beratKg`), dua metode abstrak (`hitungTarif()` dan `estimasiHari()`), serta metode biasa `ringkasan()` yang diwarisi seluruh turunan. Empat kelas moda mewarisinya melalui `extends`. Mixin `Terlacak` dipasang pada keempat moda, sedangkan mixin `Terasuransi` hanya dipasang pada kapal dan udara.



Gambar 1. Diagram hierarki kelas layanan pengiriman beserta mixin Terlacak dan Terasuransi.

4.2 lib/layanan/layanan_pengiriman.dart

Berkas ini memuat mixin `Terlacak` dan kelas abstrak `LayananPengiriman`. Mixin `Terlacak` menyimpan daftar riwayat posisi secara privat (`_riwayat`), menyediakan getter `riwayat` yang tidak dapat diubah dari luar (enkapsulasi), serta metode `catatPosisi()` dan `posisiTerakhir()`. Kelas abstrak tidak dapat dibentuk menjadi objek; ia hanya menetapkan kontrak bahwa setiap moda wajib memiliki `hitungTarif()` dan `estimasiHari()`, sedangkan `ringkasan()` sudah berisi implementasi umum sehingga tidak ditulis ulang pada tiap turunan.

```
mixin Terlacak {
  final List<String> _riwayat = [];
  List<String> get riwayat => List.unmodifiable(_riwayat);

  void catatPosisi(String lokasi) {
    final waktu = DateTime.now().toIso8601String().substring(11, 16);
    _riwayat.add('[$waktu] $lokasi');
  }

  String posisiTerakhir() =>
      _riwayat.isEmpty ? 'Belum ada data posisi' : _riwayat.last;
}

abstract class LayananPengiriman {
  final String kodeLayanan;
  final double jarakKm;
  final double beratKg;

  LayananPengiriman({
    required this.kodeLayanan,
    required this.jarakKm,
    required this.beratKg,
  });

  /// Wajib diisi oleh setiap moda pengiriman.
  double hitungTarif();

  /// Wajib diisi oleh setiap moda pengiriman.
  int estimasiHari();

  /// Implementasi umum yang diwarisi seluruh turunan.
  String ringkasan() {
    final tarif = hitungTarif().toStringAsFixed(0);
    return '$kodeLayanan | ${jarakKm.toStringAsFixed(0)} km | '
        '${beratKg.toStringAsFixed(1)} kg | Rp$tarif | ${estimasiHari()} hari';
  }
}
```

4.3 lib/layanan/moda.dart

Berkas ini memuat empat kelas turunan. Konstruktor memakai `super.kodeLayanan`, `super.jarakKm`, dan `super.beratKg` sehingga atribut bersama tidak ditulis ulang. Setiap kelas menimpa `hitungTarif()` dan `estimasiHari()` dengan `@override`. `PengirimanKeretaApi` dan `PengirimanUdara` juga menimpa `ringkasan()` dan memanggil `super.ringkasan()` agar informasi tambahan (rute stasiun dan berat volumetrik) ditempelkan pada ringkasan bawaan. Mixin `Terasuransi` dideklarasikan di berkas yang sama dan dipasang melalui `with` hanya pada kapal dan udara.

```
import 'layanan_pengiriman.dart';

mixin Terasuransi {
  double persentasePremi = 2; // dalam persen

  double hitungPremi(double nilaiBarang) => nilaiBarang * persentasePremi / 100;
}

class PengirimanTruk extends LayananPengiriman with Terlacak {
  final bool ekspres;

  PengirimanTruk({
```

```
    required super.kodeLayanan,
    required super.jarakKm,
    required super.beratKg,
    this.ekspres = false,
  });

  @override
  double hitungTarif() {
    const tarifDasarPerKm = 4500;
    const tarifPerKg = 1200;
    final tarif = (jarakKm * tarifDasarPerKm) + (beratKg * tarifPerKg);
    return ekspres ? tarif * 1.5 : tarif;
  }

  @override
  int estimasiHari() {
    final hari = (jarakKm / 450).ceil();
    return ekspres ? (hari / 2).ceil() : hari;
  }
}

class PengirimanKeretaApi extends LayananPengiriman with Terlacak {
  final String stasiunAsal;
  final String stasiunTujuan;

  PengirimanKeretaApi({
    required super.kodeLayanan,
    required super.jarakKm,
    required super.beratKg,
    required this.stasiunAsal,
    required this.stasiunTujuan,
  });

  @override
  double hitungTarif() {
    const tarifPerKmKg = 95;
    const biayaBongkarMuat = 250000;
    return (jarakKm * beratKg * tarifPerKmKg / 100) + biayaBongkarMuat;
  }

  @override
  int estimasiHari() => (jarakKm / 700).ceil() + 1;

  @override
  String ringkasan() => '${super.ringkasan()} | $stasiunAsal-$stasiunTujuan';
}

class PengirimanKapal extends LayananPengiriman with Terlacak, Terasuransi {
  final double volumeM3;
  final bool petiKemas;

  PengirimanKapal({
    required super.kodeLayanan,
    required super.jarakKm,
    required super.beratKg,
    required this.volumeM3,
    this.petiKemas = true,
  });

  @override
  double hitungTarif() {
    const tarifPerM3 = 850000;
    const biayaPelabuhan = 1750000;
    final tarif = (volumeM3 * tarifPerM3) + biayaPelabuhan;
    return petiKemas ? tarif : tarif * 1.2;
  }

  @override
  int estimasiHari() => (jarakKm / 320).ceil() + 3;
}

class PengirimanUdara extends LayananPengiriman with Terlacak, Terasuransi {
  final double panjangCm;
  final double lebarCm;
  final double tinggiCm;
```

```
  PengirimanUdara({
    required super.kodeLayanan,
    required super.jarakKm,
    required super.beratKg,
    required this.panjangCm,
    required this.lebarCm,
    required this.tinggiCm,
  });

  /// Berat volumetrik (kg) = p x l x t / 5000
  double get beratVolumetrik => (panjangCm * lebarCm * tinggiCm) / 5000;

  /// Berat yang ditagih adalah yang terbesar antara berat aktual dan volumetrik.
  double get beratTagih => beratKg > beratVolumetrik ? beratKg : beratVolumetrik;

  @override
  double hitungTarif() {
    const tarifPerKg = 15000;
    const biayaBandara = 950000;
    return (beratTagih * tarifPerKg) + biayaBandara;
  }

  @override
  int estimasiHari() => (jarakKm / 900).ceil() + 1;

  @override
  String ringkasan() =>
      '${super.ringkasan()} | Berat volumetrik: ${beratVolumetrik.toStringAsFixed(1)} kg';
}
```

4.4 latihan/modul05_uji.dart

Daftar layanan bertipe `List<LayananPengiriman>` berisi lima objek dari empat kelas turunan. Perulangan `for`, `sort`, dan `reduce` hanya memanggil `ringkasan()`, `hitungTarif()`, dan `estimasiHari()` melalui tipe induk, sehingga kode pemanggil tidak perlu tahu kelas turunan yang sebenarnya. Karena daftar sudah diurutkan berdasarkan tarif, `whereType<PengirimanTruk>().first` menghasilkan TRK-REG (tarif truk terendah) yang kemudian dipakai untuk menguji mixin `Terlacak`. Bagian terakhir menguji mixin `Terasuransi` pada kapal dan udara serta memeriksa bahwa truk bukan `Terasuransi`.

```
import '../lib/layanan/layanan_pengiriman.dart';
import '../lib/layanan/moda.dart';

void main() {
  final List<LayananPengiriman> daftarLayanan = [
    PengirimanTruk(kodeLayanan: 'TRK-REG', jarakKm: 780, beratKg: 1200),
    PengirimanTruk(
        kodeLayanan: 'TRK-EXP', jarakKm: 780, beratKg: 1200, ekspres: true),
    PengirimanKeretaApi(
      kodeLayanan: 'KA-CARGO',
      jarakKm: 780,
      beratKg: 1200,
      stasiunAsal: 'Kiaracondong',
      stasiunTujuan: 'Pasar Turi',
    ),
    PengirimanKapal(
      kodeLayanan: 'KPL-PETI',
      jarakKm: 1450,
      beratKg: 1200,
      volumeM3: 8.5,
    ),
    // Moda baru (Latihan E): hanya satu objek ditambahkan ke daftar
    PengirimanUdara(
      kodeLayanan: 'UDR-CARGO',
      jarakKm: 780,
      beratKg: 80,
      panjangCm: 100,
      lebarCm: 100,
      tinggiCm: 100,
    ),
  ];
```

```
  print('=== Perbandingan Layanan Pengiriman ===');
  for (final layanan in daftarLayanan) {
    print(layanan.ringkasan());
  }

  daftarLayanan.sort((a, b) => a.hitungTarif().compareTo(b.hitungTarif()));
  print('\nLayanan termurah : ${daftarLayanan.first.kodeLayanan}');
  print('Layanan tercepat : '
      '${daftarLayanan.reduce((a, b) => a.estimasiHari() <= b.estimasiHari() ? a : b).kodeLayanan}');

  // Pemanfaatan mixin Terlacak
  final truk = daftarLayanan.whereType<PengirimanTruk>().first;
  truk.catatPosisi('Gudang Utama Bandung');
  truk.catatPosisi('Rest Area KM 207');
  truk.catatPosisi('Gudang Transit Semarang');

  print('\nRiwayat posisi ${truk.kodeLayanan}:');
  for (final r in truk.riwayat) {
    print(' $r');
  }
  print('Posisi terakhir: ${truk.posisiTerakhir()}');

  // Tugas Praktikum: pengujian mixin Terasuransi
  print('\n=== Pengujian Mixin Terasuransi ===');
  final kapal = daftarLayanan.whereType<PengirimanKapal>().first;
  final udara = daftarLayanan.whereType<PengirimanUdara>().first;
  const nilaiBarang = 10000000.0;
  print('Persentase premi kapal: ${kapal.persentasePremi.toStringAsFixed(0)}%');
  print('Premi kapal untuk nilai barang Rp${nilaiBarang.toStringAsFixed(0)}: '
      'Rp${kapal.hitungPremi(nilaiBarang).toStringAsFixed(0)}');
  print('Persentase premi udara: ${udara.persentasePremi.toStringAsFixed(0)}%');
  print('Premi udara untuk nilai barang Rp${nilaiBarang.toStringAsFixed(0)}: '
      'Rp${udara.hitungPremi(nilaiBarang).toStringAsFixed(0)}');

  print('\n=== Pengujian PengirimanTruk ===');
  final LayananPengiriman layananTruk = truk;
  if (layananTruk is Terasuransi) {
    print('PengirimanTruk memiliki metode hitungPremi().');
  } else {
    print('PengirimanTruk tidak memiliki metode hitungPremi().');
  }
}
```

5. Hasil Eksekusi dan Analisis

5.1 Pengujian Polimorfisme dan Mixin Terlacak (Praktikum)

```
PS C:\Users\ACER\Documents\silog-714240045\Pertemuan 5\flutter_application_1\silog_app> dart run .\latihan\modul05_uji.dart
=== Perbandingan Layanan Pengiriman ===
TRK-REG | 780 km | 1200.0 kg | Rp4950000 | 2 hari
TRK-EXP | 780 km | 1200.0 kg | Rp7425000 | 1 hari
KA-CARGO | 780 km | 1200.0 kg | Rp1139200 | 3 hari | Kiaracondong-Pasar Turi
KPL-PETI | 1450 km | 1200.0 kg | Rp8975000 | 8 hari


Layanan termurah : KA-CARGO
Layanan tercepat : TRK-EXP

Riwayat posisi TRK-REG:
 [18:01] Gudang Utama Bandung
 [18:01] Rest Area KM 207
 [18:01] Gudang Transit Semarang
Posisi terakhir: [18:01] Gudang Transit Semarang
```

Gambar 2. Hasil dart run latihan/modul05_uji.dart untuk empat moda awal (truk reguler, truk ekspres, kereta api, kapal).

Metode `ringkasan()` dipanggil pada tiap elemen `List<LayananPengiriman>` dan menghasilkan empat baris dengan tarif yang berbeda, walaupun jarak (780 km) dan berat (1200 kg) tiga layanan pertama sama. Mixin `Terlacak` bekerja: tiga posisi dicatat pada TRK-REG dan `posisiTerakhir()` mengembalikan Gudang Transit Semarang.

5.2 Perbandingan tarif dan penyebab perbedaannya

Layanan	Rumus tarif (kelas masing-masing)	Tarif	Estimasi
TRK-REG	(780 × 4500) + (1200 × 1200) = 3.510.000 + 1.440.000	Rp4.950.000	ceil(780/450) = 2 hari
TRK-EXP	Tarif truk reguler × 1,5 (ekspres)	Rp7.425.000	ceil(2/2) = 1 hari
KA-CARGO	(780 × 1200 × 95 / 100) + 250.000 = 889.200 + 250.000	Rp1.139.200	ceil(780/700) + 1 = 3 hari
KPL-PETI	(8,5 × 850.000) + 1.750.000 = 7.225.000 + 1.750.000 (peti kemas, tanpa pengali 1,2)	Rp8.975.000	ceil(1450/320) + 3 = 8 hari

- **TRK-REG** dihitung dari jarak dan berat secara terpisah, sehingga tarifnya tumbuh sejalan dengan keduanya. Truk cepat untuk jarak menengah, tetapi biaya per kilometernya tinggi.
- **TRK-EXP** memakai rumus yang sama dengan TRK-REG lalu dikalikan 1,5. Tarif naik 50% sebagai imbalan estimasi waktu yang dipangkas separuh (2 hari menjadi 1 hari), sehingga ini layanan tercepat.
- **KA-CARGO** adalah yang termurah. Tarif per km-kg sangat kecil (95/100) dan hanya ditambah biaya bongkar muat tetap Rp250.000. Harga murahnya dibayar dengan waktu: 3 hari karena ada tambahan 1 hari untuk proses stasiun.
- **KPL-PETI** adalah yang termahal pada daftar awal. Tarifnya ditentukan volume (8,5 m^3) dan biaya pelabuhan tetap, bukan berat atau jarak, dan jaraknya 1450 km sehingga paling lama (8 hari, termasuk 3 hari penanganan pelabuhan).

Layanan termurah dan tercepat ditentukan oleh kode yang sama dengan sebelumnya, yaitu hanya membandingkan hasil `hitungTarif()` dan `estimasiHari()` tanpa memeriksa jenis kelasnya.

5.3 Sifat kelas abstrak

Ketika baris `final x = LayananPengiriman(kodeLayanan: 'X', jarakKm: 1, beratKg: 1);` ditulis, Dart menolaknya dengan pesan `Abstract classes can't be instantiated`. Pesan ini menjelaskan sifat kelas abstrak: ia hanya kerangka kontrak, bukan cetakan objek utuh. Karena `hitungTarif()` dan `estimasiHari()` tidak memiliki badan, objek langsung dari kelas ini tidak akan tahu cara menghitung tarifnya. Objek hanya dapat dibentuk dari turunan konkret yang sudah mengisi semua metode abstrak. Baris percobaan ini dihapus setelah pengamatan.

5.4 Penambahan Moda Baru (Latihan)

```
PS C:\Users\ACER\Documents\silog-714240045\Pertemuan 5\flutter_application_1\silog_app> dart run .\latihan\modul05_uji.dart
=== Perbandingan Layanan Pengiriman ===
TRK-REG | 780 km | 1200.0 kg | Rp4950000 | 2 hari
TRK-EXP | 780 km | 1200.0 kg | Rp7425000 | 1 hari
KA-CARGO | 780 km | 1200.0 kg | Rp1139200 | 3 hari | Kiaracondong-Pasar Turi
KPL-PETI | 1450 km | 1200.0 kg | Rp8975000 | 8 hari
UDR-CARGO | 780 km | 80.0 kg | Rp3950000 | 2 hari | Berat volumetrik: 200.0 kg


Layanan termurah : KA-CARGO
Layanan tercepat : TRK-EXP

Riwayat posisi TRK-REG:
 [18:05] Gudang Utama Bandung
 [18:05] Rest Area KM 207
 [18:05] Gudang Transit Semarang
Posisi terakhir: [18:05] Gudang Transit Semarang
```

Gambar 3. Hasil dart run latihan/modul05_uji.dart setelah PengirimanUdara (UDR-CARGO) ditambahkan.

Baris UDR-CARG0 muncul pada hasil dengan berat aktual 80,0 kg dan berat volumetrik 200,0 kg. Karena berat volumetrik lebih besar, yang ditagih adalah 200 kg: (200 × 15.000) + 950.000 = **Rp3.950.000** dengan estimasi ceil(780/900) + 1 = 2 hari. Layanan termurah (KA-CARGO) dan tercepat (TRK-EXP) tetap sama.

Bukti pemanggil tidak berubah. Perbedaan Gambar 2 dan Gambar 3 hanya satu baris keluaran baru. Pada `modul05_uji.dart` hanya satu objek `PengirimanUdara` yang ditambahkan ke `daftarLayanan`; perulangan `for`, `sort`, `reduce`, dan pemanggilan `ringkasan()` tidak diubah sama sekali. Hal ini dimungkinkan oleh **polimorfisme**: kode pemanggil bergantung pada tipe induk `LayananPengiriman` (kontrak `hitungTarif()`, `estimasiHari()`, `ringkasan()`), bukan pada kelas turunan tertentu. Dart memilih implementasi yang sesuai dengan kelas objek sebenarnya saat metode dipanggil, sehingga moda baru cukup memenuhi kontrak tersebut. Prinsipnya terbuka untuk penambahan, tertutup untuk perubahan.

5.5 Pengujian Mixin Terasuransi (Tugas)

```
PS C:\Users\ACER\Documents\silog-714240045\Pertemuan 5\flutter_application_1\silog_app> dart run .\latihan\modul05_uji.dart

=== Perbandingan Layanan Pengiriman ===
TRK-REG | 780 km | 1200.0 kg | Rp4950000 | 2 hari
TRK-EXP | 780 km | 1200.0 kg | Rp7425000 | 1 hari
KA-CARGO | 780 km | 1200.0 kg | Rp1139200 | 3 hari | Kiaracondong-Pasar Turi
KPL-PETI | 1450 km | 1200.0 kg | Rp8975000 | 8 hari
UDR-CARGO | 780 km | 80.0 kg | Rp3950000 | 2 hari | Berat volumetrik: 200.0 kg

Layanan termurah : KA-CARGO
Layanan tercepat : TRK-EXP

Riwayat posisi TRK-REG:
 [18:08] Gudang Utama Bandung
 [18:08] Rest Area KM 207
 [18:08] Gudang Transit Semarang
Posisi terakhir: [18:08] Gudang Transit Semarang

=== Pengujian Mixin Terasuransi ===
Persentase premi kapal: 2%
Premi kapal untuk nilai barang Rp10000000: Rp200000
Persentase premi udara: 2%
Premi udara untuk nilai barang Rp10000000: Rp200000

=== Pengujian PengirimanTruk ===
PengirimanTruk tidak memiliki metode hitungPremi().
```

Gambar 4. Hasil pengujian mixin Terasuransi pada PengirimanKapal dan PengirimanUdara, serta pengujian PengirimanTruk.

Persentase premi kapal dan udara masing-masing 2%. Untuk nilai barang Rp10.000.000, premi yang dihitung `hitungPremi()` adalah Rp200.000 pada kedua moda (10.000.000 × 2 / 100). Pada bagian terakhir, objek truk diperiksa dengan `is Terasuransi` dan hasilnya false, sehingga yang tercetak adalah *PengirimanTruk tidak memiliki metode hitungPremi()*. Ini membuktikan mixin hanya melekat pada kelas yang memasangnya.

6. Jawaban Tugas Praktikum

- **Membuat mixin Terasuransi.** Mixin `Terasuransi` menyediakan atribut `persentasePremi` (bernilai 2 persen) dan metode `hitungPremi(double nilaiBarang)` yang mengembalikan `nilaiBarang * persentasePremi / 100`.
- **Memasang mixin hanya pada kapal dan udara.** Mixin dipasang dengan `with Terlacak, Terasuransi` pada `PengirimanKapal` dan `PengirimanUdara`. `PengirimanTruk` dan `PengirimanKeretaApi` tidak memasangnya. Program uji membuktikan bahwa truk tidak memiliki `hitungPremi()` (Gambar 4).
- **Alasan memakai mixin, bukan pewarisan.** Truk, kereta api, kapal, dan udara berhubungan "adalah sebuah" layanan pengiriman, sehingga dimodelkan dengan pewarisan dari kelas abstrak. Kemampuan dilacak dan diasuransikan berhubungan "memiliki kemampuan" dan tidak dimiliki semua moda. Jika `hitungPremi()` diletakkan di kelas induk, truk dan kereta api ikut memilikinya padahal tidak perlu. Dart juga tidak mendukung pewarisan berganda, sedangkan mixin boleh dipasang lebih dari satu pada satu kelas (kapal dan udara memakai dua mixin sekaligus).

- **Alasan memakai kelas abstrak.** `hitungTarif()` dan `estimasiHari()` wajib dimiliki semua moda tetapi caranya berbeda, sehingga cukup dideklarasikan sebagai metode abstrak. `ringkasan()` sama untuk semua moda sehingga diwarisi langsung tanpa pengulangan kode.

Tabel skenario uji

No	Skenario	Input	Output yang diharapkan	Output yang diperoleh	Status
1	Pewarisan dan override tarif	dart run latihan/modul05_uji.dart (empat moda)	Tarif tiap moda berbeda sesuai rumusnya	TRK-REG Rp4.950.000; TRK-EXP Rp7.425.000; KA-CARGO Rp1.139.200; KPL-PETI Rp8.975.000	Sesuai
2	Polimorfisme melalui tipe induk	Sort dan reduce pada List<LayananPengiriman>	Termurah dan tercepat terdeteksi	Termurah KA-CARGO; tercepat TRK-EXP	Sesuai
3	Mixin Terlacak	catatPosisi() tiga kali pada TRK-REG	Tiga riwayat tercatat	Tiga baris riwayat; posisi terakhir Gudang Transit Semarang	Sesuai
4	Instansiasi kelas abstrak	LayananPengiriman(...) langsung	Ditolak compiler	Error: Abstract classes can't be instantiated	Sesuai
5	Penambahan PengirimanUdara	Satu objek UDR-CARGO ditambah ke daftar	Baris baru muncul, kode pemanggil tidak diubah	UDR-CARGO Rp3.950.000, 2 hari, berat volumetrik 200,0 kg	Sesuai
6	Mixin Terasuransi pada kapal dan udara	hitungPremi(10000000)	Premi 2% = Rp200.000	Rp200000 untuk kapal dan udara	Sesuai
7	Truk tanpa Terasuransi	Pemeriksaan is Terasuransi pada truk	Truk tidak memiliki hitungPremi()	PengirimanTruk tidak memiliki metode hitungPremi().	Sesuai

7. Kesimpulan dan Kendala

Kesimpulan. Praktikum Modul 05 berhasil dilaksanakan dan seluruh skenario uji menghasilkan keluaran sesuai harapan. Pewarisan dari kelas abstrak `LayananPengiriman` menghilangkan pengulangan atribut dan metode `ringkasan()` pada tiap moda, sedangkan penggantian metode membuat tiap moda memiliki tarif dan estimasi waktu sesuai karakteristiknya. Polimorfisme terbukti: moda baru `PengirimanUdara` ditambahkan tanpa mengubah kode pemanggil. Mixin `Terlacak` dan `Terasuransi` memungkinkan perilaku tambahan dibagikan hanya pada kelas yang membutuhkannya, dan terbukti truk tidak memiliki `hitungPremi()`.

Kendala. Pada kode akhir tidak ditemukan error. Hal yang memerlukan ketelitian adalah penulisan `@override` dengan tanda tangan metode yang tepat, pemasangan mixin dengan urutan `extends` sebelum `with`, serta pemilihan berat yang ditagih pada moda udara (berat aktual atau volumetrik, yang terbesar). Selain itu, urutan `daftarLayanan` berubah setelah `sort`, sehingga objek truk untuk uji mixin diambil dengan `whereType` dan bukan dengan indeks.

8. Tautan Repositori dan Riwayat Commit

Repositori GitHub: https://github.com/Nogi3201/silog-714240045

Cabang: https://github.com/Nogi3201/silog-714240045/tree/main/Pertemuan%205 (cabang kerja lokal: praktikum/modul-05)

Riwayat commit: Modul 5: Pewarisan, Polimorfisme, Kelas Abstrak, dan Mixin